

Gauge + Playwright + Docker — Interview Cheat Sheet

Interview: 30.07.2026, 15:00 Uhr — Michael Lucas (Consultant Manager) & Micha Schulz (Sales Manager)

Ziel: Frontend-Testautomatisierung mit Gauge (Spec) + Playwright (Automatisierung) + Docker (reproduzierbare Umgebung), live nachbaubar.

1. Setup-Sequenz (von Null zum laufenden Framework)

```
node -v
npm -v

npm install -g @getgauge/cli
gauge version

gauge init ts

npm uninstall taiko
npm install playwright
npx playwright install chromium
npm install csv-parse

gauge run specs
# oder: npm test

docker build -t gauge-playwright-demo .
docker run --rm gauge-playwright-demo

git init
git add .
git commit -m "... "
git remote add origin <repo-url>
git push -u origin master # Branch-Name im Workflow-Trigger muss passen!
```

Windows-Stolperstein (falls `gauge version` ein Fenster aufblitzen lässt und nichts tut): Leere Platzhalter-Datei `gauge` (ohne Endung) liegt in `%AppData%\npm\node_modules\@getgauge\cli\bin\` NEBEN der echten `gauge.exe`. Fix: **Rename-Item** auf die leere Datei, dann findet Windows automatisch `gauge.exe`.

tsconfig-Stolperstein: `"ignoreDeprecations"` akzeptiert in TS 5.9 NUR den exakten String `"5.0"` (die Warnmeldung schlägt irreführend `"6.0"` vor). Zusätzlich `"skipLibCheck": true` setzen — sonst schlägt `npx tsc --noEmit` in `playwright-cores` eigenen `.d.ts`-Dateien fehl (fehlende DOM-Typen, weil `lib` kein `"dom"` enthält).

2. Datengetriebene Specs — das zentrale Gauge-Feature

Eine Tabelle DIREKT unter der `###`-Überschrift (nicht unter einem einzelnen Step!) macht das ganze Szenario darunter datengetrieben: Gauge wiederholt es automatisch einmal PRO ZEILE und ersetzt `<spalte>` in jedem Step — kein `Table`-Objekt im Code nötig. Neue Testdaten = neue Zeile, kein Code nötig.

Talking Point: Reale Drittanbieter-Suchmaschinen (Google, DuckDuckGo) live in Tests anzusteuern ist KEIN gutes Beispiel — beide zeigen CAPTCHA-Challenges bei Bot-Traffic (live getestet: DuckDuckGo → "Select all squares containing a duck"). Externe Abhängigkeiten mocken oder gegen eigene, kontrollierte Umgebung testen.

3. Context- und Teardown-Steps

Gauge-Spec-Konzept	Läuft wann	Code-Äquivalent
Context Steps (unter <code>#</code> , vor erstem <code>##</code>)	Vor JEDEM Szenario	<code>@BeforeScenario()</code>
Teardown Steps (nach <code>___</code> , am Dateiende)	Nach JEDEM Szenario	<code>@AfterScenario()</code>

Code-Hooks für technisches Plumbing (Browser-Seite öffnen/schließen, für Test Manager uninteressant). Context/Teardown-Steps IN der Spec für fachlich relevantes Setup/Cleanup (Login/Logout, Screenshot). Bei mehreren Datenzeilen läuft Teardown MEHRFACH — einmal pro Zeile/Szenario-Ausführung (live nachgewiesen: `delete_todo.spec` mit 2 Zeilen → 2 Screenshots).

Jede Zeile in einer Spec, die keine Überschrift/`tags:`/Step/Tabellenzeile/`___`-Trenner ist, wird als reiner Kommentar/Dokumentation behandelt — nie ausgeführt, aber im Report sichtbar.

4. Jira-Rückverfolgbarkeit + Tag-Konvention

Kategorie	Beispiel	Zweck
Jira-Rückverfolgbarkeit	<code>jira-QAT-101</code>	Verknüpfung zum Jira-Testfall/Story (fiktive Platzhalter-IDs)
Fachliches Modul	<code>todo, add, complete, delete, csv, database</code>	Funktionsbereich
Test-Typ	<code>smoke, regression</code>	Testebene/-zweck
Priorität	<code>P1, P2</code>	Kritikalität

```
gauge run -t "smoke" specs
gauge run -t "jira-QAT-102" specs
```

5. Externe Datenquellen: CSV und Datenbank

Gauges native Tabellen-Syntax funktioniert NUR mit Markdown-Tabellen direkt in der `.spec`-Datei — nicht mit CSV/DB. Für externe Quellen schreibt man einen Step, der selbst liest (`loadCsvRows()`) bzw.

`getAllTodoDescriptions()` in `tests/StepImplementation.ts/tests/db.ts`). Muster: Testdaten kommen von außen (Testmanager/Fachbereich pflegt CSV oder DB), Step-Code bleibt gleich einfach.

SQLite-Wahl: `node:sqlite` (in Node 22+ eingebaut, kein npm-Paket, kein natives Kompilieren). Verifiziert: läuft lokal (Node 22.14) UND im Docker-Image (`mcr.microsoft.com/playwright:v1.62.0-noble` bringt Node 24.18 mit). DB liest Testdaten (`todos`-Tabelle) UND schreibt Testergebnisse zurück (`test_results`-Tabelle, via `@AfterScenario`-Hook mit `ExecutionContext` → `getCurrentScenario().getIsFailing()`).

DB ansehen: VS-Code-Extension "SQLite Viewer" (`qwtel.sqlite-viewer`) installiert — `.db`-Datei anklicken öffnet Tabellen-Ansicht. Oder CLI:

```
& "$env:LOCALAPPDATA\Android\Sdk\platform-tools\sqlite3.exe" testdata\gauge.db
.tables "SELECT * FROM test_results;"
```

6. Dockerfile

```
FROM mcr.microsoft.com/playwright:v1.62.0-noble
WORKDIR /app
RUN npm install -g @getgauge/cli && gauge install ts
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["gauge", "run", "specs"]
```

Versionsnummer im Image-Tag **muss** zur `playwright`-Version in `package.json` passen.

7. GitHub Actions (`.github/workflows/ci.yml`)

```
name: Gauge Playwright Tests
on:
  push: { branches: [ master ] }
  pull_request: { branches: [ master ] }
  schedule:
    - cron: '0 2 * * *' # täglich 02:00 UTC – Drift unabhängig von Code-
    Änderungen abfangen
  workflow_dispatch: # manuelles Auslösen im Actions-Tab
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t gauge-playwright-demo .
      - run: docker run --rm -v ${GITHUB_WORKSPACE}/reports:/app/reports
        gauge-playwright-demo
      - if: always()
```

```
uses: actions/upload-artifact@v4
with: { name: gauge-html-report, path: reports/html-report }
```

Branch im Trigger muss zum Default-Branch passen. Alle drei Trigger-Arten (push/schedule/workflow_dispatch) live verifiziert.

8. Nützliche Befehle (Cheat-Referenz)

Befehl	Zweck
<code>gauge run specs</code>	Alle Specs ausführen
<code>gauge run specs/delete_todo.spec:5</code>	Ein Szenario nach Zeilennummer
<code>gauge run --scenario "Name" specs/delete_todo.spec</code>	Ein Szenario nach Name
<code>gauge run -t "tagname" specs</code>	Nach Tag filtern
<code>gauge run -f</code>	Nur zuletzt fehlgeschlagene Szenarien
<code>gauge run -v</code>	Verbose (Step-Level statt Szenario-Level)
<code>\$env:HEADLESS="false"; gauge run ...</code>	Browser sichtbar (PowerShell-Syntax!)
<code>npx playwright codegen <url></code>	Locators durch Klicken generieren lassen
<code>npx tsc --noEmit</code>	TypeScript-Sanity-Check vor dem Testlauf

9. Kernkonzepte — Talking Points

- **Gauge trennt WAS von WIE:** Spec (Markdown) vs. Step-Implementierung (Code) — Anknüpfungspunkt für Sales-Sicht.
- **Datengetriebene Specs:** Zeile = unabhängiger Testfall, neue Testdaten ohne Code-Änderung.
- **Playwright ist nur Library, nicht Runner:** `playwright`, nicht `@playwright/test` — Gauge ist der Runner.
- **Locators statt CSS-Selektoren:** `getByRole`, `getByTestId`, `getByPlaceholder`.
- **Auto-Waiting:** kein `sleep()`, Playwright wartet auf Actionability.
- **isVisible() prüft display/Bounding-Box, nicht opacity** — TodoMVC-Löschbutton braucht daher echten `.hover()` (selbst verifiziert).
- **Docker = reproduzierbare Umgebung**, gleicher Befehl lokal wie in CI.
- **Typsicherheit endet an der Spec-Grenze:** `gauge-tss PrimitiveParser` konvertiert Zahlen/Booleans automatisch, aber ungültige Werte ("`abc`") erzeugen stille Fehler (`NaN` → `setTimeout(NaN)` läuft durch) — deshalb Parameter an der Grenze explizit validieren.
- **Zwei Tabellen-Muster nicht verwechseln:** Inline-Step-Tabelle vs. datengetriebene Spec.
- **Externe Datenquellen (CSV/DB):** Step liest selbst, kein Gauge- Kernfeature nötig; externe Live-Abhängigkeiten (Suchmaschinen) meiden.
- **Context/Teardown-Steps** laufen pro Szenario, nicht einmal pro Datei.
- **Gauge generiert automatisch HTML-Reports** — als CI-Artefakt archivierbar.

10. Für die Sales-Frage ("Wie erklärst du einem Kunden den Nutzen?")

- Gauge-Specs sind lesbares Markdown → Product Owner/Kunden verstehen Testabdeckung ohne Code zu lesen.
 - Docker: "funktioniert auf meiner Maschine" ist kein Thema mehr.
 - CI/CD verhindert kaputte Änderungen in Produktion — automatisch bei jedem Push, nachts, oder manuell auslösbar.
-

11. Eigene Referenzprojekte

- github.com/Albamaris/ecommerce-test-automation — Page Object Pattern, GitHub Actions + Jenkins
- github.com/Albamaris/interview_systemverification — dieses Demo-Framework (heute gebaut: Gauge + Playwright + Docker + CI/CD + Jira-Tags + CSV/DB-Anbindung)